

ALGORITHM DESIGN AND ANALYSIS

UNIT-1 :- Divide and Conquer

Algorithm	Time Complexity	Space Complexity
Binary Search	Best case : $O(1)$ Average case : $O(\log n)$ Worst case : $O(\log n)$ (where , n = number of elements)	$O(1)$
Quick Sort	Best case: $O(n \log n)$ Average case: $O(n \log n)$ Worst case : $O(n^2)$ (where , n = number of items)	Best case : $O(\log n)$ Average case: $O(\log n)$ Worst case : $O(n)$ (where , n = number of items)
Merge Sort	Best case: $O(n \log n)$ Average case: $O(n \log n)$ Worst case : $O(n \log n)$ (where , n = number of items)	$O(n)$ (where , n = number of items)
Strassen's Matrix Multiplication	$O(n^{\log_2 7}) \approx O(n^{2.807})$ (where , n = the dimension of the square matrices being multiplied)	$O(n^2)$ (where , n = the dimension of the square matrices being multiplied)

UNIT-2 :- Disjoint Sets and Backtracking

Algorithm	Time Complexity	Space complexity
Disjoint Sets - Union and Find	Best case: $O(1)$ Average case: $O(\alpha(n))$ Worst case: $O(\alpha(n))$ (where , $\alpha(n)$ = inverse ackermann function)	$O(n)$ (where, n = number of elements)
Priority Queue	Best case: $O(1)$ Average case: $O(\log n)$ Worst case: $O(\log n)$ (where , n = number of elements)	$O(n)$ (where n = number of elements)
Heap - Build Heap	$O(n)$ (where , n = number of elements)	$O(n)$ (where , n = number of elements)
Heap - Insert/Delete	Best case: $O(1)$ Average case: $O(\log n)$ Worst case: $O(\log n)$ (where , n = number of elements)	$O(n)$ (where, n = number of elements)
Heap Sort	$O(n \log n)$ (where , n = number of elements)	$O(1)$ auxiliary
Backtracking - General Method	Worst case: $O(b^d)$ (where , b = maximum number of choices available at decision point d =maximum depth of the recursion tree)	$O(d)$ (where , d = maximum depth of the recursion tree)
N-Queens Problem	Worst case: $O(n!)$ (where , n = number of queens/size of the chessboard)	$O(n)$ (where , n = number of queens/ size of the chessboard)

Sum of Subsets	Worst case : $O(2^n)$ (where , n = total number of elements)	$O(n)$ (where, n = total number of elements)
Graph coloring	Worst case : $O(m^n)$ (where n = number of vertices, m = number of colors)	$O(n)$ (where, n = number of vertices)
Hamiltonian Cycles	Worst case : $O(n!)$ (where , n = number of vertices)	$O(n)$ (where , n = number of vertices)

UNIT-3 :- Dynamic Programming

Algorithm	Time Complexity	Space Complexity
Optimal Binary Search Tree (OBST)	$O(n^3)$	$O(n^2)$
0/1 Knapsack	$O(nW)$ (Where, n= number of items w= max weight capacity)	$O(nW)$ (Where n= number of items w= weight capacity)
All-Pairs Shortest Path (Floyd–Warshall)	$O(n^3)$ (where, n = number of vertices)	$O(n^2)$ (where, n = number of vertices)
Traveling Salesperson Problem (TSP)	$O(n^2 2^n)$ (where , n= number of cities)	$O(n 2^n)$ (where, n= number of cities)
Reliability Design	$O(nC)$ (where, n =number of devices C =maximum cost constraint)	$O(nC)$ (where, n =number of devices C =maximum cost constraint)

UNIT-4 : - Greedy method

Algorithm	Time Complexity	Space Complexity
Greedy – General Method	$O(n \log n)$ (where, n =Number of elements/nodes/items in the input)	$O(n)$ (where, n =Number of elements/nodes/items in the input)
Job Sequencing with Deadlines	Best Case: $O(n \log n)$ Worst Case: $O(n^2)$ (where, n =Number of elements/nodes/items in the input)	$O(n)$ (where, n =Number of elements/nodes/items in the input)
Fractional Knapsack	$O(n \log n)$ (where, n =Number of elements/nodes/items in the input)	$O(n)$ (where, n =Number of elements/nodes/items in the input)
Minimum Cost Spanning Tree – Prim's(Binary Heap + Adjacency List)	$O(E \log V)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)	$O(V + E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)
Minimum Cost Spanning Tree – Prim's(Adjacency Matrix)	$O(V^2)$ (where, v =Number of vertices (nodes) in a graph)	$O(V^2)$ (where, v =Number of vertices (nodes) in a graph)
Minimum Cost Spanning Tree – Prim's(Fibonacci Heap + Adjacency List)	$O(E + V \log V)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)	$O(V + E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)
Minimum Cost Spanning Tree – Kruskal's	$O(E \log E) \approx O(E \log V)$ (where, e =Number of edges in a graph, v =Number of vertices)	$O(V + E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)

	(nodes) in a graph)	
Single Source Shortest Path – Dijkstra's Algorithm	$O((V+E) \log V)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)	$O(V + E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)
Single Source Shortest Path – (Fibonacci Heap + Adjacency List)	$O(E + V \log V)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)	$O(V + E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)
Single Source Shortest Path – (Adjacency Matrix)	$O(V^2)$ (where, v =Number of vertices (nodes) in a graph)	$O(V^2)$ (where, v =Number of vertices (nodes) in a graph)
Binary Tree Traversal - (DFS / BFS)	$O(n)$ (where, n =Number of elements/nodes/items in the input)	DFS: $O(h)$; BFS: $O(n)$ (where, n =Number of elements/nodes/items in the input, h =Height of a tree)
Graph Traversals (DFS/BFS)	$O(V+E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)	$O(V)$ (where, v =Number of vertices (nodes) in a graph)
Connected Components	$O(V+E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)	$O(V)$ (where, v =Number of vertices (nodes) in a graph)
Biconnected Components	$O(V+E)$ (where, e =Number of edges in a graph, v =Number of vertices (nodes) in a graph)	$O(V)$ (where, v =Number of vertices (nodes) in a graph)

--	--	--

UNIT-5 :- Branch and Bound

Algorithm	Time Complexity	Space Complexity
0/1 Knapsack – LC (Least-Cost)	Best case: $O(n \log n)$ Average case: $O(2^n)$ Worst case: $O(2^n)$ (where, n = number of items)	Best case: $O(n)$ Average case: $O(2^n)$ Worst case: $O(2^n)$ (where, n = number of items)
0/1 Knapsack – FIFO	Best case: $O(n)$ Average case: $O(2^n)$ Worst case: $O(2^n)$ (where, n = number of items)	Best case: $O(n)$ Average case: $O(2^n)$ Worst case: $O(2^n)$ (where, n = number of items)
Traveling Salesperson Problem -General Method	Best case: $O(n^2)$ Average case: $O(n!)$ Worst case: $O(n!)$ (where, n = number of cities)	Best case: $O(n^2)$ Average case: $O(n!)$ Worst case: $O(n!)$ (where, n = number of cities)
Traveling Salesperson Problem (Branch and Bound) – FIFO	$O(n!)$ (where, n = number of cities)	$O(n!)$ (where, n = number of cities)